



Python 모듈



1. 에러 처리
2. 파일 읽고 쓰기
3. 모듈과 패키지
4. Gemini 앱 만들기

▼ 1. 에러 처리



에러란 프로그램 실행 중에 발생하는 예외적인 상황을 말합니다.

`try - except` 문을 사용하면 안전하게 에러를 처리할 수 있습니다.

▼ 에러의 종류

에러 종류	설명	예시
문법 에러 (Syntax Error)	프로그램의 구문이 잘못된 경우	들여쓰기 오류, 괄호 닫기 누락

런타임 에러 (Runtime Error)	프로그램 실행 중에 발생하는 에러	없는 파일 접근, 0으로 나누기
ValueError	부적절한 값이 사용될 때 발생	문자열을 정수로 변환 시도
TypeError	잘못된 타입의 연산을 수행할 때 발생	숫자와 문자열의 덧셈
IndexError	리스트 등의 인덱스 범위를 벗어날 때 발생	존재하지 않는 인덱스 접근
KeyError	딕셔너리에서 존재하지 않는 키를 참조할 때 발생	없는 키 접근

▼ 예외 처리 구문

```
try:
    # 에러가 발생할 수 있는 코드
except 에러 종류1:
    # 에러 발생시 실행할 코드
except 에러 종류2 as 변수명:
    # 에러 발생시 실행할 코드
finally:
    # 무조건 실행되는 코드
```

- **try 블록:** 에러가 발생할 수 있는 코드를 포함하는 블록입니다.
- **except 블록:** 특정 에러가 발생했을 때 실행되는 코드를 포함합니다.
- **except as 구문:** 발생한 에러의 정보를 변수로 받아서 사용할 수 있습니다. (예: `except ValueError as e`)
- **finally 블록:** 에러 발생 여부와 관계없이 항상 실행되는 코드를 포함합니다.

▼ 예제 코드

```
def divide_numbers(a, b):
    try:
        print(a / b)
    except ZeroDivisionError:
        print("0으로 나눌 수 없습니다!")
    except TypeError as e:
        print(e)
    finally:
        print("함수 실행 완료")
```

```
divide_numbers(10, 2)    # 정상 실행
divide_numbers(10, 0)    # ZeroDivisionError
divide_numbers(10, '2')  # TypeError
```

▼ 직접 에러 발생시키기

- `raise` 구문을 사용하면 특정 조건에서 **의도적으로 에러를 발생시킬** 수 있습니다.
- 예외적인 상황이 발생했을 때, 에러를 발생시키거나 프로그램을 중단시키고 싶을 때 사용합니다.

```
def check_age(age):
    if age < 0:
        raise ValueError("나이는 음수가 될 수 없습니다!")
    print(f"입력된 나이: {age}")

try:
    check_age(-5)
except ValueError as e:
    print(e)
```

▼ 실습 과제

▼ 실습1

이전 시간에 배운 BankAccount 문제를 **예외처리 문법**을 활용하여 다시 작성해보십시오.

- 입금한 금액에 오류가 있을 때, ValueError를 사용하여 에러를 발생시킵니다.
- 출금하려는 금액에 오류가 있을 때, ValueError를 사용하여 에러를 발생시킵니다.
- 잔액 조회 시 오류가 있을 때, 에러를 발생시킵니다.
- 계좌 정보 조회 시 오류가 있을 때, 에러를 발생시킵니다.

▼ 실습2

다음과 같은 사용자 정보 검증 프로그램을 작성해보세요:

```
def validate_user_info(username, age, password):
    # 여기에 코드를 작성하세요

# 테스트 케이스
test_cases = [
    ("user1", 25, "password123"),      # 정상 케이스
    ("a", -5, "pwd"),                  # 모든 필드가
    잘못된 경우                        # 모든 필드가
    ("user2!!", 150, "onlytext"),      # 모든 필드가
    잘못된 경우                        # 잘못된 나이
    ("gooduser", "invalid", "pwd123")  타입
]

# text에 숫자가 포함 되어있는지 체크하는 함수
def check_digit_contained(text):
    has_digit = any(char.isdigit() for char in text)
    return has_digit

# text에 문자가 포함 되어있는지 체크하는 함수
def check_char_contained(text):
    has_letter = any(char.isalpha() for char in password)
    return has_letter
```

▼ 요구사항

1. 각 필드별로 적절한 검증 로직을 구현하세요.

조건

- **username:** 3자 이상 20자 이하
 - **age:** 0살 초과 120살 이하의 정수
 - **password:** 8자 이상, 숫자와 문자 모두 포함
2. 잘못된 입력에 대해 구체적인 에러 메시지를 포함하는 예외를 발생시키세요.

4. 모든 테스트 케이스에 대해 적절한 예외 처리가 되어야 합니다.

▼ 힌트

숫자와 문자가 모두 포함되어 있는지 확인:

```
def check_password_requirements(password):
    # 숫자가 포함되어 있는지
    has_digit = any(char.isdigit() for char in password)
    # 문자가 포함되어 있는지
    has_letter = any(char.isalpha() for char in password)

    return has_digit and has_letter
```

▼ 2. 파일 읽고 쓰기



파일 입출력은 데이터 분석, 로그 처리, 설정 관리 등에 중요한 작업입니다.

기본적인 파일 처리

- 파일 열기/닫기: `open()` 함수와 `with` 문을 사용합니다.
- 파일 모드: 'r'(읽기), 'w'(쓰기), 'a'(추가), 'b'(바이너리)

```
# 기본적인 파일 읽기/쓰기
with open('example.txt', 'r', encoding='utf-8') as f:
    content = f.read()
    print(content)

with open('example.txt', 'w', encoding='utf-8') as f:
    f.write('안녕하세요\n')
    f.write('파일 쓰기 예제입니다.')
```

▼ 실습: 텍스트 파일을 활용한 간단한 일기장 만들기

```

class Diary:
    def __init__(self, filename='my_diary.txt'):
        self.filename = filename

    def write(self):
        date = input("날짜를 입력하세요 (YYYY-MM-DD): ")
        content = input("오늘의 일기를 작성하세요: ")
        # 여기에 코드를 완성해주세요

    def read(self):
        # 여기에 코드를 완성해주세요

```

▼ 코드 예시

```

class Diary:
    def __init__(self, filename='my_diary.txt'):
        self.filename = filename

    def write(self):
        try:
            date = input("날짜를 입력하세요 (YYYY-MM-DD): ")
            content = input("오늘의 일기를 작성하세요: ")

            with open(self.filename, 'a', encoding='utf-8') as f:
                f.write(f"\n=== {date} ===\n")
                f.write(f"{content}\n")
                print("일기가 성공적으로 저장되었습니다.")
        except Exception as e:
            print(f"일기 작성 중 오류가 발생했습니다: {e}")

    def read(self):
        try:
            with open(self.filename, 'r', encoding

```

```

='utf-8') as f:
    content = f.read()
    if content.strip():
        print("\n=== 일기장 내용 ===")
        print(content)
    else:
        print("일기장이 비어있습니다.")
except FileNotFoundError:
    print("일기장 파일이 존재하지 않습니다.")
except Exception as e:
    print(f"일기 읽기 중 오류가 발생했습니다: {e}")

# 사용 예시
diary = Diary()
while True:
    print("\n1. 일기 쓰기")
    print("2. 일기 읽기")
    print("3. 종료")
    choice = input("원하는 작업을 선택하세요 (1-3): ")

    if choice == '1':
        diary.write()
    elif choice == '2':
        diary.read()
    elif choice == '3':
        print("프로그램을 종료합니다.")
        break
    else:
        print("잘못된 선택입니다. 다시 선택해주세요.")

```

일기를 쓰고 읽을 수 있는 **Diary** 클래스를 완성해주세요.

1. **write**: 일기를 쓰고 저장하는 기능
2. **read**: 이전에 작성한 일기를 읽어오는 기능



`input()` 함수를 사용하면 사용자로부터 직접 입력을 받아 일기를 작성할 수 있습니다.

▼ 3. 모듈과 패키지

모듈과 패키지는 여러 Python 코드와 파일들을 묶어놓은 꾸러미입니다.

▼ 모듈이란?

- 하나의 파이썬 파일(`.py`)로서, 특정 기능을 하는 파이썬 함수나 클래스들을 묶어둔 파이썬 코드입니다.

▼ 모듈 불러오는 법

`from` 과 `import` 키워드를 사용하면 모듈을 내 코드에 불러올 수 있습니다.

사용 방법	문법	설명
기본 임포트	<code>import module_name</code>	모듈 전체를 가져와서 <code>module_name.function()</code> 형태로 사용
별칭 사용	<code>import module_name as alias</code>	모듈에 별칭을 지정하여 <code>alias.function()</code> 형태로 사용
특정 함수만 임포트	<code>from module_name import function</code>	필요한 함수만 가져와서 직접 <code>function()</code> 형태로 사용
여러 함수 임포트	<code>from module_name import func1, func2</code>	여러 함수를 선택적으로 가져와서 사용
전체 함수 임포트	<code>from module_name import *</code>	모든 함수를 직접 사용 가능 (권장하지 않음)

```
import math
```

```
print(math.pi)           # 3.141592...
```

```
print(math.sqrt(16))     # 4.0
```



```
# 특정 함수만 임포트
from random import randint

print(randint(1, 10)) # 1~10 사이의 난수
```

▼ 패키지란?

- 여러 파이썬 파일들(모듈들)을 포함하는 **일종의 폴더**입니다. '라이브러리'라고 하는 이름으로도 불립니다.
- 책으로 비유하면, 각각의 한 페이지를 모듈이라고 할 수 있고, 여러 페이지를 묶은 한 권의 책이 패키지라고 할 수 있습니다.

▼ 패키지 불러오는 법

패키지 불러오기	예시	설명
기본 임포트	<code>import package_name</code>	패키지 전체를 가져와서 사용
하위 모듈 임포트	<code>from package_name import module</code>	특정 하위 모듈만 가져오기
하위 패키지 임포트	<code>from package_name.subpackage import module</code>	하위 패키지의 모듈 가져오기
별칭 사용	<code>import package_name as pkg</code>	패키지에 별칭을 지정하여 사용

```
# 기본 임포트
import matplotlib.pyplot
matplotlib.pyplot.plot([1, 2, 3])

# 별칭 사용
import matplotlib.pyplot as plt
plt.plot([1, 2, 3])

# 특정 함수만 임포트
from matplotlib.pyplot import plot, title
plot([1, 2, 3])
title("그래프 제목")

# 하위 패키지의 모듈 임포트
```

```
from matplotlib import pyplot
pyplot.plot([1, 2, 3])

# * 사용 (권장하지 않음)
from matplotlib.pyplot import *
plot([1, 2, 3])
```

▼ 자주 사용되는 표준 모듈

- **math**: 수학 관련 함수와 상수
- **random**: 난수 생성 관련 기능
- **datetime**: 날짜와 시간 처리
- **os**: 운영체제 관련 기능

```
import os

# 현재 작업 디렉토리 출력
print(os.getcwd())
```

▼ 외부 패키지 받아오는 법



파이썬 표준 라이브러리에 포함되지 않은 외부 패키지도 받아올 수 있습니다.

▼ pip 사용하기

`pip` 는 파이썬의 패키지 관리자로, 외부 패키지를 설치하는 데 사용됩니다.

- **기본 설치 명령어:** `pip install package_name`
- **특정 버전 설치:** `pip install package_name==version`
- **패키지 업그레이드:** `pip install --upgrade package_name`
- **패키지 제거:** `pip uninstall package_name`

```
# NumPy 설치 예시
pip install numpy

# pandas 특정 버전 설치
pip install pandas==2.1.0

# matplotlib 최신 버전으로 업그레이드
pip install --upgrade matplotlib
```

▼ 패키지 검색 사이트

- **PyPI (Python Package Index):** <https://pypi.org> - 파이썬 공식 패키지 저장소

▼ 자주 사용되는 외부 패키지

- **NumPy:** 수치 계산을 위한 패키지
- **pandas:** 데이터 분석을 위한 패키지
- **matplotlib:** 데이터 시각화 패키지
- **scikit-learn:** 머신러닝 패키지
- **requests:** HTTP 통신을 위한 패키지

▼ 4. Gemini 앱 만들기

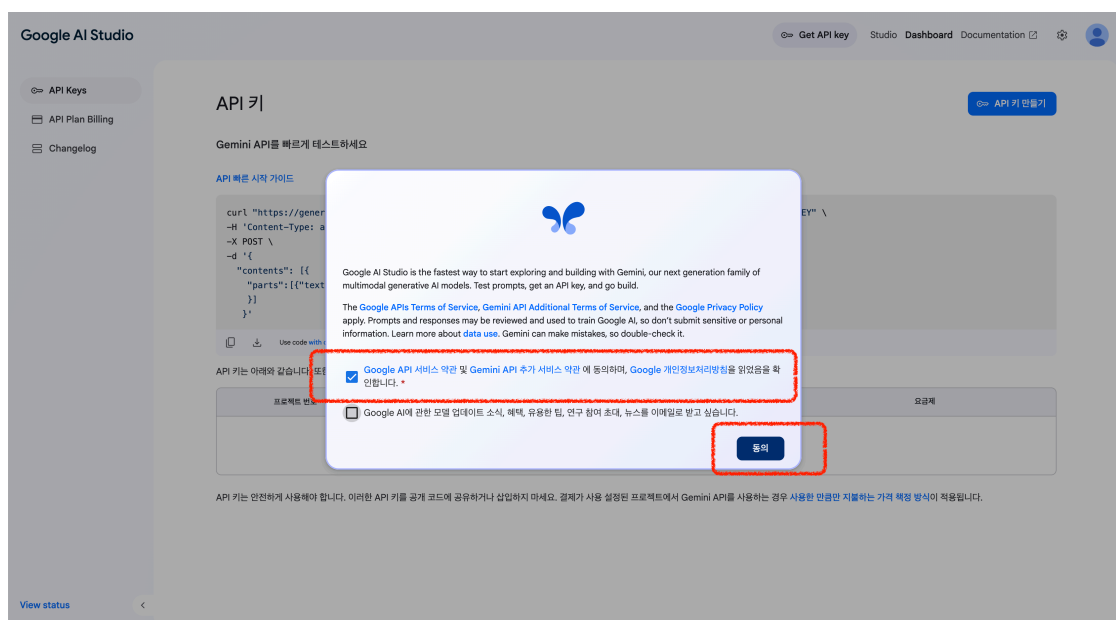


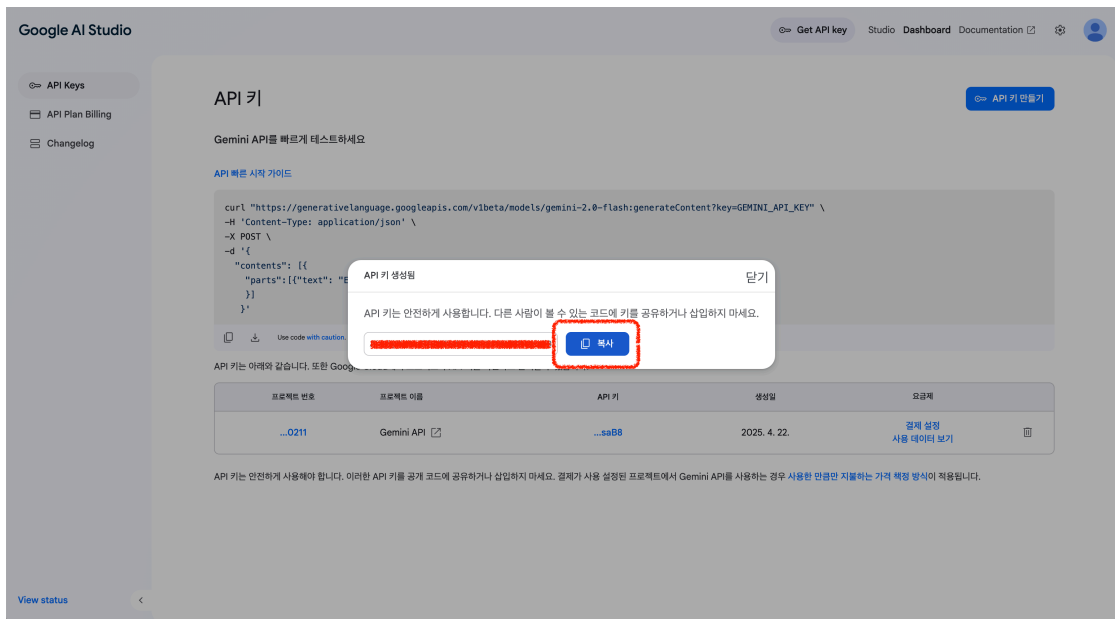
▼ Gemini란?

- 구글에서 만든 대화형 인공지능 모델
- 무료 버전으로 손쉽게 API 연동 가능

▼ Step0: Google AI Studio에서 API를 발급 받습니다.

<https://aistudio.google.com/app/apikey>





마지막에 생성된 API는 꼭 잊지 말고! 남들이 함부로 볼 수 없는 안전한 곳에 복사해둡니다!

▼ Step1: 패키지 설치하기

Gemini를 사용하기 위한 패키지를 설치해야 합니다. 아래 명령어를 실행해주세요.
설치가 다 되신 분들은 **Kernel > Restart** 를 실행해주세요.



pip 명령어를 사용하면 외부 패키지를 설치할 수 있습니다.

```
pip install google-generativeai
```

▼ Step2: 패키지 가져오기

Gemini API를 사용하기 위해 필요한 패키지를 가져옵니다.



`import 패키지명 as 별명` 을 사용하면 패키지명을 가져오되, 더 짧은 이름으로 사용할 수 있게 됩니다.

```
import google.generativeai as genai
```

▼ Step3: API 키 설정하기

API 키는 우리가 Google의 Gemini 서비스를 사용할 수 있는 인증 키입니다.

Step0 에서 발급받은 API 키를 코드에 입력해주세요.



API 키는 외부 서비스와 통신할 때 개발자의 신원을 인증하는 중요한 수단입니다.

API 키는 절대 공개된 코드에 포함시키지 않도록 주의하세요!!

```
# Gemini API 키를 여기에 입력하세요
```

```
API_KEY = "YOUR_API_KEY_HERE"
```

```
# API_KEY를 사용하여 genai를 초기화 합니다.
```

```
genai.configure(api_key=API_KEY)
```

▼ Step4: Gemini 모델 생성하기

`GenerativeModel` 클래스를 이용해 Gemini 모델을 생성합니다. `gemini-2.0-flash` 는 무료로 사용 가능한 LLM 모델입니다.



클래스를 사용하면 실제 동작하는 객체를 생성할 수 있습니다.

`GenerativeModel` 클래스를 사용하여 실제로 질문을 주고 받을 수 있는 객체를 생성할 수 있습니다.

```
# Gemini 모델 생성
model = genai.GenerativeModel("gemini-2.0-flash")
```

▼ Step5: 사용자 입력 받기

`input()` 함수를 사용하여 사용자로부터 질문을 입력받습니다.

`input()` 함수를 사용하면 사용자로부터 입력을 받을 수 있습니다.
입력된 값은 항상 문자열 형태로 반환됩니다.

```
# 사용자 입력 받기
user_input = input("무엇이든 질문해보세요: ")
```

▼ Step6: Gemini에 질문 보내기

사용자의 질문을 Gemini 모델에 전송하고 응답을 받습니다.

`generate_content()` 메서드를 호출하면 Gemini 서버로 질문이 전송되고 답변을 받아옵니다.

```
# Gemini에 질문 보내기
response = model.generate_content(user_input)
```

▼ Step 7: 응답 출력하기

Gemini의 응답을 콘솔에 출력합니다.

Gemini의 응답 객체에서 `text` 속성을 사용하면 답변 내용을 가져올 수 있습니다.

```
# 응답 출력
print(response.text)
```

▼ 퀴즈: 질문을 계속 주고 받으려면?

질문과 답변을 여러 번 주고 받게 하려면 어떻게 바꿔볼 수 있을까요?

▼ 전체 코드

```
import google.generativeai as genai

while True:
    # Gemini API 키를 여기에 입력하세요
    API_KEY = "API 키를 여기에 입력하세요"

    # API_KEY를 사용하여 genai를 초기화 합니다.
    genai.configure(api_key=API_KEY)

    # Gemini 모델 생성
    model = genai.GenerativeModel("gemini-2.0-flash")

    # 사용자 입력 받기
    user_input = input("무엇이든 질문해보세요: ")

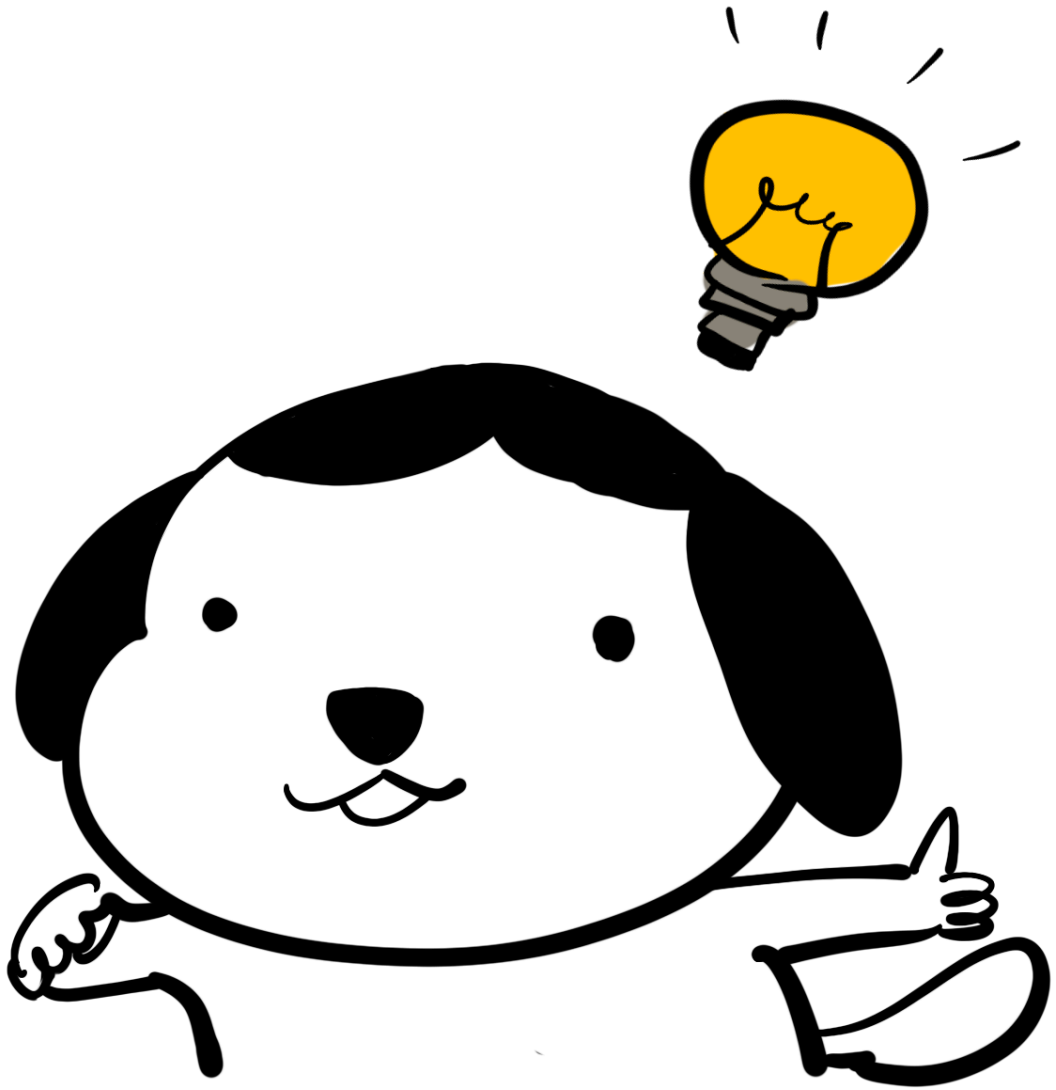
    # Gemini에 질문 보내기
    response = model.generate_content(user_input)
    print(response.text)
```

▼ 제3회 LG전자 DXSchool배 인공지능 사생대회



▼ 다음 강아지 캐릭터의 이름을 지어봅시다.

#댕댕이 #병맛넘치게 #개발자



▼ 다음 페이지에 기고 & 투표해봅시다.

Reetro.io

Reetro

 <https://www.reetro.app/board/670e6f4009714f0bd3b5a568/686fdcf7d04b06ac0b9b1e1c>